

ASSIGNMENT REPORT

Digital Design

Assignment 4.0 - 16-bit ALU

Course	Digital Design
Assignment	Assignment 4.0 - 16-bit ALU
Student Name	Mohamed Hossam El-Sawy
Instructor / TA	Eng. Ali Tamsah
Submission Date	July 20, 2026

01 OBJECTIVE

This assignment designs, implements and verifies a 16-bit Arithmetic Logic Unit (ALU_TOP) that forms a fundamental building block of a processor datapath. The ALU carries out signed arithmetic, Boolean logic, comparison and shift operations, selected by a 4-bit function code (ALU_FUNC) and enabled through a 2x4 decoder that reads the two most-significant bits of that code.

02 PROBLEM STATEMENT / REQUIREMENTS

The ALU_TOP module must be built from five sub-blocks - ARITHMETIC_UNIT, LOGIC_UNIT, CMP_UNIT, SHIFT_UNIT and a Decoder Unit - and verified with a self-checking testbench covering 31 test cases. Key requirements:

- Six separate Verilog modules: Arithmetic_Unit, Logic_Unit, Cmp_Unit, Shift_Unit, Decoder_Unit and ALU_TOP.
- ALU_FUNC[3:2] selects the active block (Arithmetic / Logic / Compare / Shift) through the 2x4 decoder; ALU_FUNC[1:0] selects the operation inside that block.
- All outputs are registered and cleared using an asynchronous active-low reset (RST).
- Each block asserts its own flag (Arith_Flag, Logic_Flag, CMP_Flag, SHIFT_Flag) high only while that block is active.
- Arithmetic operands and results are signed (two's complement); Arith_OUT is a 32-bit product/quotient-capable output.

- A self-checking testbench exercises 28+ directed test cases (later extended to 31) covering every sign combination of add/sub/mul/div, all four logic functions, all compare outcomes and all four shift operations.
- Simulation clock: 100 kHz with a 40% low / 60% high duty cycle.

03 DESIGN & ANALYSIS

3.1 Design Approach

ALU_TOP is structured as a top-level module instantiating a Decoder_Unit and four functional units in parallel (Arithmetic_Unit, Logic_Unit, Cmp_Unit, Shift_Unit). The Decoder_Unit is a combinational 2-to-4 decoder that reads ALU_FUNC[3:2] and drives one of four block-enable signals (Arith_Enable, Logic_Enable, Cmp_Enable, Shift_Enable) high at a time. Each functional unit only updates its registered output when its own enable is asserted; otherwise the output and flag are held/cleared, matching the case/if-else pattern suggested in the assignment hints. Arithmetic operands (A, B) and the Arith_OUT/Arith_Flag path are declared as signed so Verilog performs true two's-complement addition, subtraction, multiplication and division.

3.2 Truth Tables

Decoder Unit Truth Table - ALU_FUNC[3:2] selects the active functional block:

ALU_FUNC[3:2]	Arith_En	Logic_En	CMP_En	SHIFT_En
00	1	0	0	0
01	0	1	0	0
10	0	0	1	0
11	0	0	0	1

Table 1: Decoder Unit truth table

ALU_FUN Operation Table - ALU_FUNC[1:0] selects the operation within the active block:

ALU_FUN	Operation
0000	Arithmetic: Signed Addition
0001	Arithmetic: Signed Subtraction
0010	Arithmetic: Signed Multiplication
0011	Arithmetic: Signed Division
0100	Logic: AND
0101	Logic: OR
0110	Logic: NAND
0111	Logic: NOR
1000	Compare: NOP -> 0
1001	Compare: A = B -> 1 else 0
1010	Compare: A > B -> 2 else 0
1011	Compare: A < B -> 3 else 0
1100	Shift: A >> 1
1101	Shift: A << 1
1110	Shift: B >> 1
1111	Shift: B << 1

Table 2: ALU_FUN operation table

Top-level (ALU_TOP) port list:

Signal Name	Width (bits)
A	parameterized (16)
B	parameterized (16)
ALU_FUNC	4
CLK	1
RST	1
Arith_OUT	parameterized (32)
Arith_Flag	1
Logic_OUT	parameterized (16)
Logic_Flag	1
CMP_OUT	parameterized (16)
CMP_Flag	1
SHIFT_OUT	parameterized (16)
SHIFT_Flag	1

Table 3: ALU_TOP port description

3.3 Block Diagram

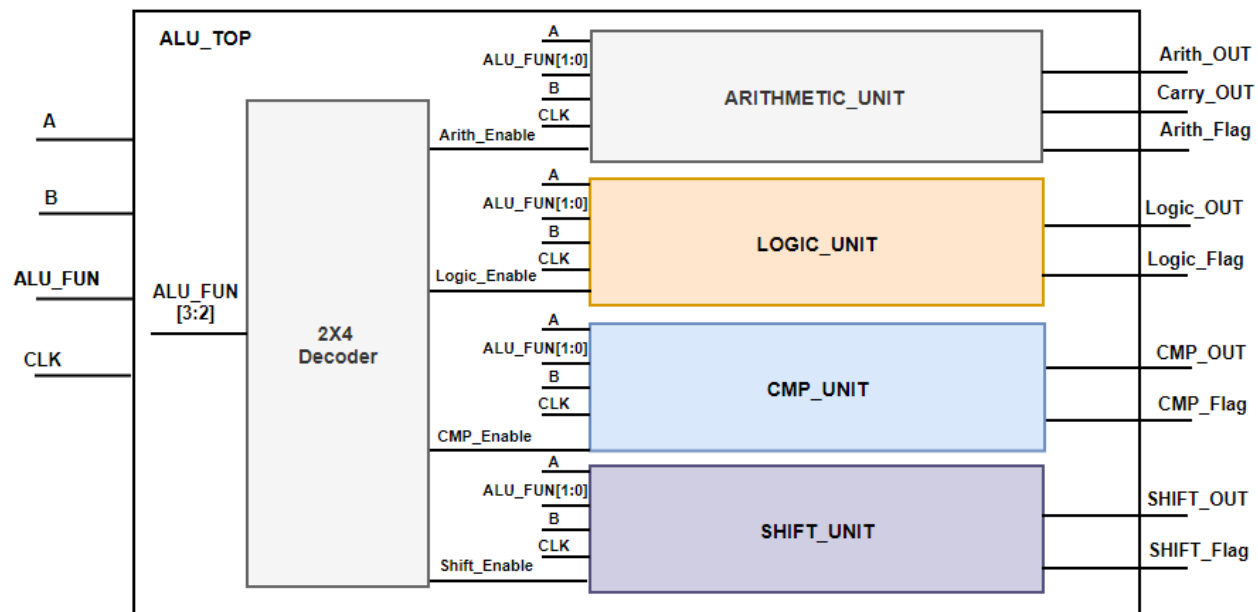


Figure 1: ALU_TOP block diagram - Decoder Unit feeds block-enable signals to the Arithmetic, Logic, Compare and Shift units

3.4 HDL Implementation

The design is split into six Verilog files. Decoder_Unit is purely combinational; the four functional units are clocked (posedge CLK) with asynchronous active-low reset, and each holds its output/flag at 0 whenever its enable is low. ALU_TOP wires everything together and passes ALU_FUNC[3:2] to the decoder and ALU_FUNC[1:0] to each functional unit. Representative snippets:

Decoder_Unit.v (combinational 2x4 decoder):

```
always @(*) begin
    Zero = 0; One = 0; Two = 0; Three = 0;
    case (Input)
```

```

2'b00: Zero = 1;
2'b01: One = 1;
2'b10: Two = 1;
2'b11: Three = 1;
default: begin Zero=0; One=0; Two=0; Three=0; end
endcase
end

```

Arithmetic_Unit.v (signed operands, registered output/flag):

```

input signed [inputWidth-1:0] A, B;
...
always @(*) begin
  if (Arith_Enable) begin
    case (ARITHMETIC_FUN)
      2'b00: Arith_OUT_D = A + B;
      2'b01: Arith_OUT_D = A - B;
      2'b10: Arith_OUT_D = A * B;
      2'b11: Arith_OUT_D = A / B;
      default: Arith_OUT_D = 'b0;
    endcase
  end else
    Arith_OUT_D = 'b0;
end

```

ALU_TOP.v (structural top level):

```

Decoder_Unit Decoder_Unit_Block (.Input(ALU_FUN[3:2]),
  .Zero(Arith_Enable), .One(Logic_Enable),
  .Two(Cmp_Enable), .Three(Shift_Enable));

Arithmetic_Unit #(inputWidth(inputWidth), .outputWidth(outputWidth*2))
  Arithmetic_Unit_Block (.A(A), .B(B), .ARITHMETIC_FUN(ALU_FUN[1:0]),
    .RST(RST), .CLK(CLK), .Arith_Enable(Arith_Enable),
    .Arith_OUT(Arith_OUT), .Arith_Flag(Arith_Flag));
// Logic_Unit, Cmp_Unit and Shift_Unit instantiated the same way

```

The full source of all six modules (Arithmetic_Unit.v, Logic_Unit.v, Cmp_Unit.v, Shift_Unit.v, Decoder_Unit.v, ALU_TOP.v) and the testbench (ALU_TOP_TB.v) accompany this report.

04 SIMULATION & RESULTS

Verification approach: ALU_TOP_TB.v is a self-checking, directed testbench that drives A, B, ALU_FUN and RST, and compares every registered output/flag pair against an expected value with \$display PASS/FAIL reporting. The design was simulated in ModelSim at a 100 kHz clock with a 40% low / 60% high duty cycle. Signed test operands were applied either as negative decimal literals or their two's-complement hex equivalent, per the assignment hints. The suite covers 31 test cases: all four sign combinations of signed addition, subtraction, multiplication and division; all four logic functions (AND, OR, NAND, NOR); NOP plus the true/false cases of equal, greater-than and less-than compare; and all four shift operations (A>>1, A<<1, B>>1, B<<1).

4.1 Test Case Summary

#	Test Case	ALU_FUN	Output	Flags	Result
1	ADD: NEG + NEG	0000	4294967282 (0xffffffff2)	0001	PASSED
2	ADD: POS + NEG	0000	10 (0x0000000a)	0001	PASSED
3	ADD: NEG + POS	0000	11 (0x0000000b)	0001	PASSED

#	Test Case	ALU_FUN	Output	Flags	Result
4	ADD: POS + POS	0000	55 (0x00000037)	0001	PASSED
5	SUB: NEG - NEG	0001	6 (0x00000006)	0001	PASSED
6	SUB: POS - NEG	0001	30 (0x0000001e)	0001	PASSED
7	SUB: NEG - POS	0001	4294967277 (0xfffffed)	0001	PASSED
8	SUB: POS - POS	0001	30 (0x0000001e)	0001	PASSED
9	MUL: NEG * NEG	0010	40 (0x00000028)	0001	PASSED
10	MUL: POS * NEG	0010	4294967286 (0xfffff6)	0001	PASSED
11	MUL: NEG * POS	0010	4294967282 (0xfffff2)	0001	PASSED
12	MUL: POS * POS	0010	48 (0x00000030)	0001	PASSED
13	DIV: NEG / NEG	0011	2 (0x00000002)	0001	PASSED
14	DIV: POS / NEG	0011	4294967281 (0xfffff1)	0001	PASSED
15	DIV: NEG / POS	0011	4294967292 (0xfffffc)	0001	PASSED
16	DIV: POS / POS	0011	25 (0x00000019)	0001	PASSED
17	LOGIC: AND	0100	0xf0	0010	PASSED
18	LOGIC: OR	0101	0xff0f	0010	PASSED
19	LOGIC: NAND	0110	0xf0ff	0010	PASSED
20	LOGIC: NOR	0111	0x00f0	0010	PASSED
21	CMP: NOP	1000	0	0100	PASSED
22	CMP: A == B (True)	1001	1	0100	PASSED
23	CMP: A == B (False)	1001	0	0100	PASSED
24	CMP: A > B (True)	1010	2	0100	PASSED
25	CMP: A > B (False)	1010	0	0100	PASSED
26	CMP: A < B (True)	1011	3	0100	PASSED
27	CMP: A < B (False)	1011	0	0100	PASSED
28	SHIFT: A >> 1	1100	0x7f80	1000	PASSED
29	SHIFT: A << 1	1101	0x01fe	1000	PASSED
30	SHIFT: B >> 1	1110	0x007f	1000	PASSED
31	SHIFT: B << 1	1111	0xfe00	1000	PASSED

Table 4: Summary of all 31 directed test cases (extracted from the simulation log) - 31/31 PASSED

4.2 Simulation Waveforms

Waveforms were captured in ModelSim, grouped by functional block. Each screenshot shows A_TB, B_TB, ALU_FUN_TB, CLK_TB, RST_TB and the four registered output/flag buses over the relevant time window.

Reset & Initialization (0 - 30 us)

Asynchronous active-low reset (RST) drives A, B, ALU_FUN and all registered outputs (Arith_OUT, Logic_OUT, Cmp_OUT, Shift_OUT) and flags to zero.

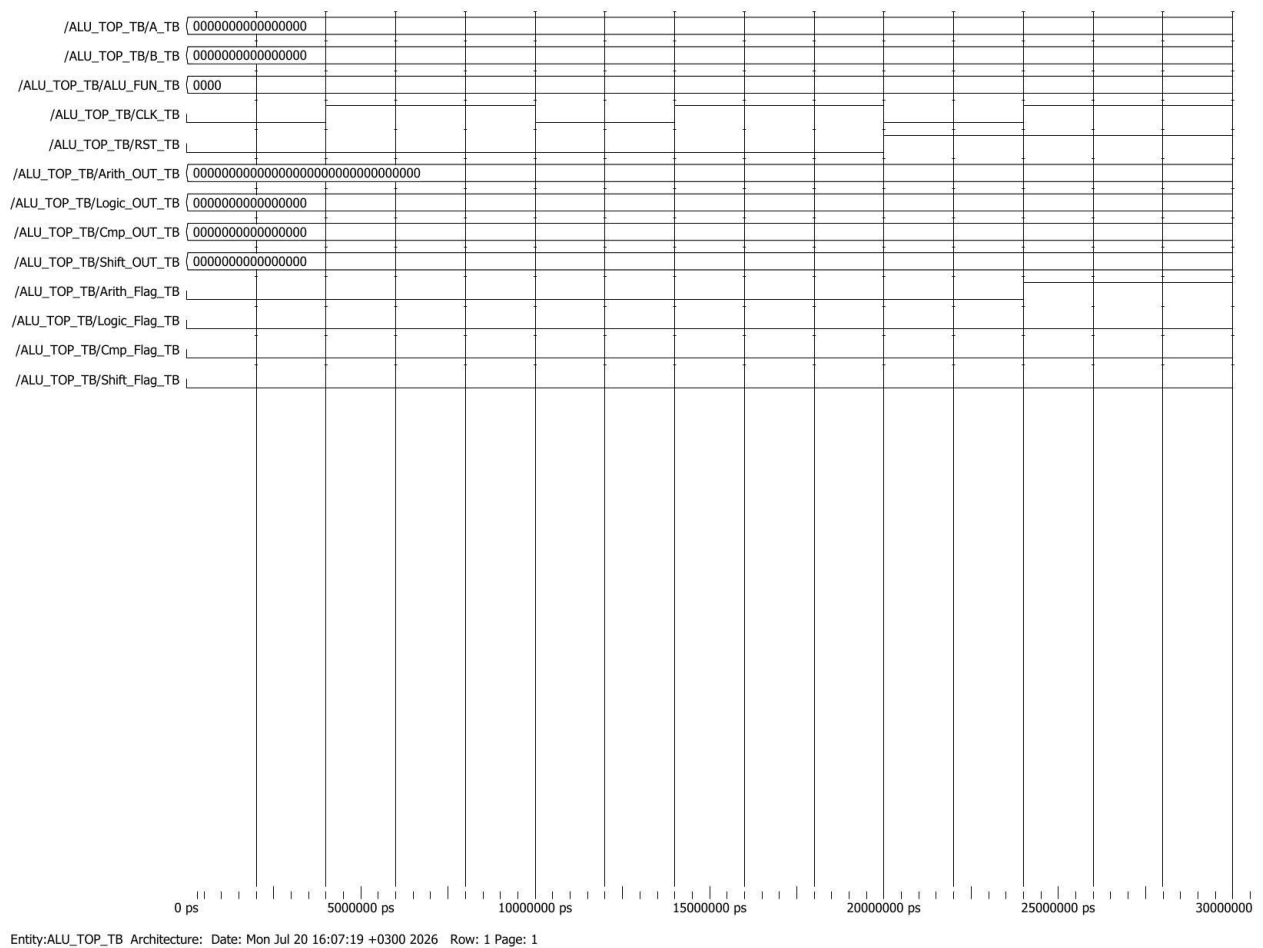
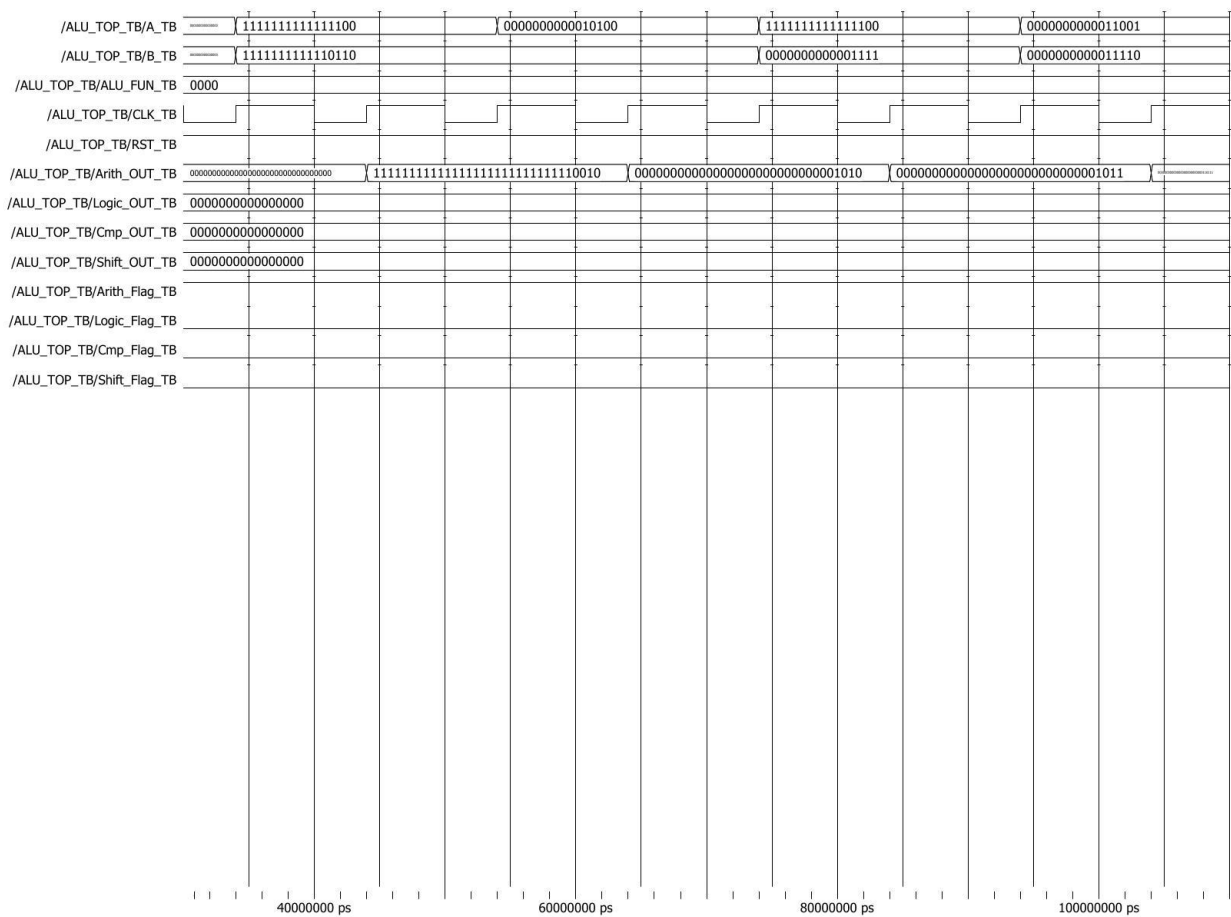


Figure 2: Reset & Initialization (0 - 30 us)

Signed Arithmetic - Addition (30 - 110 us), ALU_FUN = 0000

Covers the four sign combinations of A + B (Test 1-4). Arith_Flag = 1 while Logic/Cmp/Shift flags remain 0.

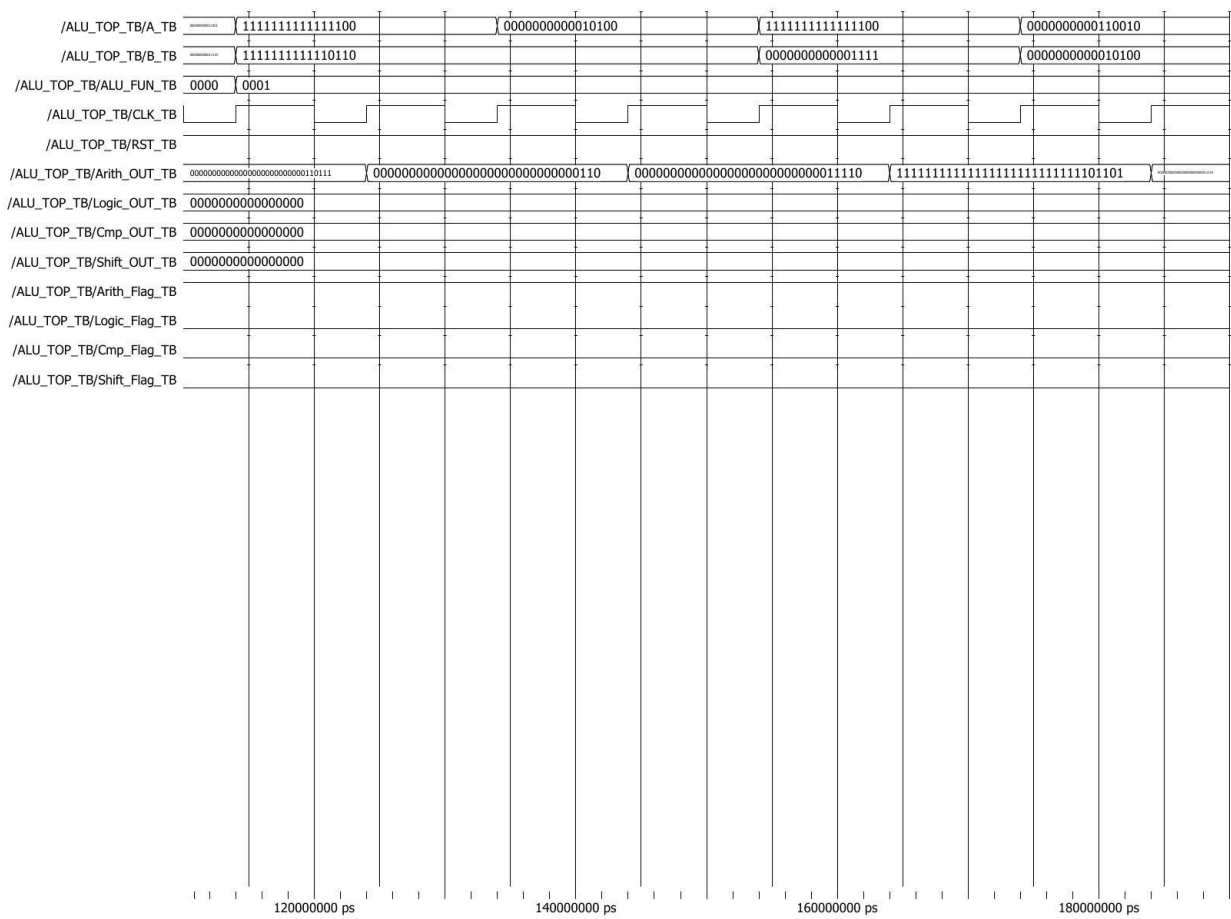


Entity:ALU_TOP_TB Architecture: Date: Mon Jul 20 16:09:06 +0300 2026 Row: 1 Page: 1

Figure 3: Signed Arithmetic - Addition (30 - 110 us), ALU_FUN = 0000

Signed Arithmetic - Subtraction (110 - 190 us), ALU_FUN = 0001

Covers the four sign combinations of A - B (Test 5-8).



Entity:ALU_TOP_TB Architecture: Date: Mon Jul 20 16:11:41 +0300 2026 Row: 1 Page: 1

Figure 4: Signed Arithmetic - Subtraction (110 - 190 us), ALU_FUN = 0001

Signed Arithmetic - Multiplication (190 - 270 us), ALU_FUN = 0010

Covers the four sign combinations of $A * B$ (Test 9-12); the 32-bit Arith_OUT captures the full product.



Signed Arithmetic - Division (270 - 350 us), ALU_FUN = 0011

Digital Design - Assignment 4.0 - 16-bit ALU | Page 9 of 14

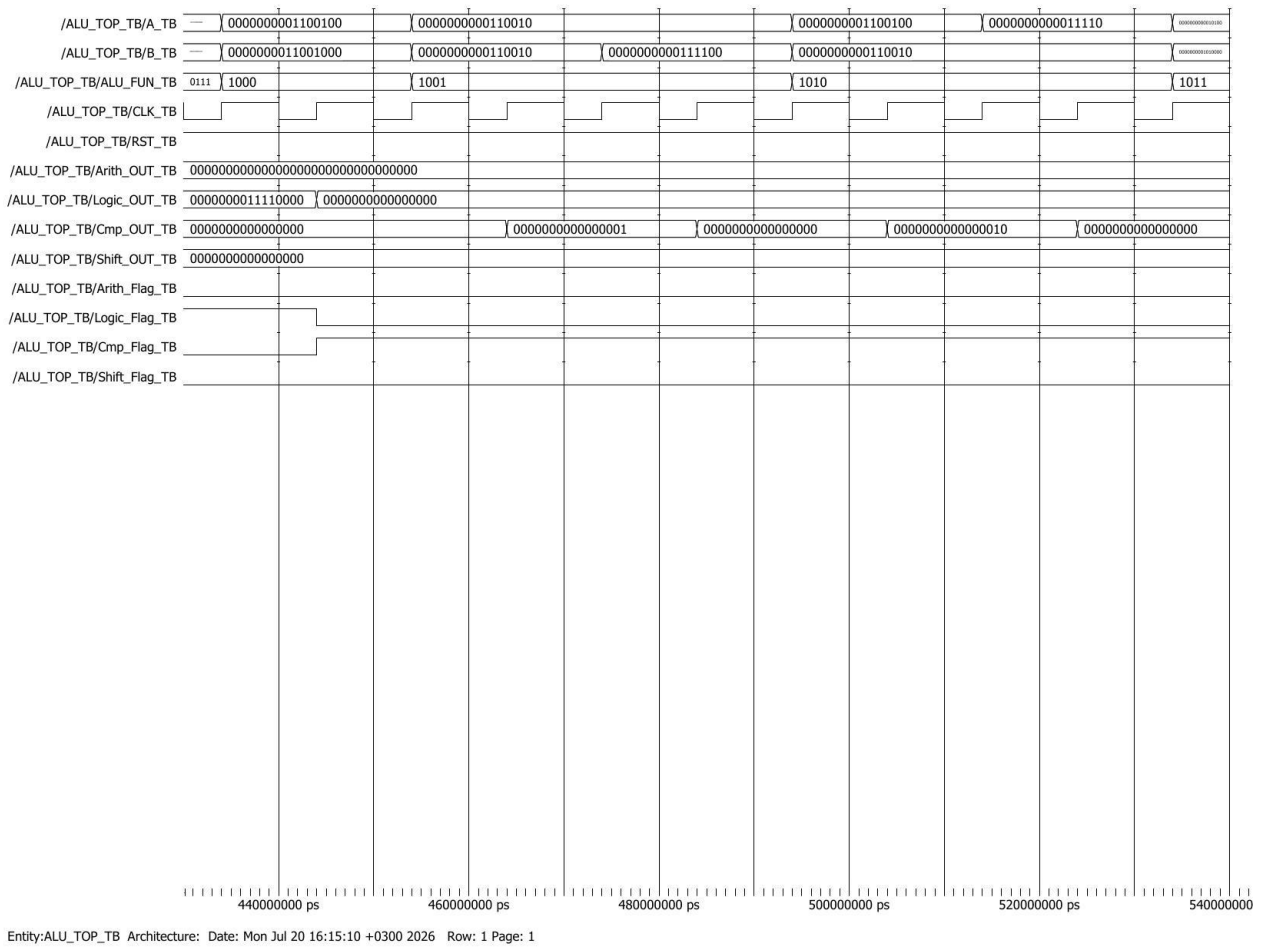


Figure 8: Compare Operations (430 - 540 us), ALU_FUN = 10xx

Shift Operations (540 - 620 us), ALU_FUN = 11xx

A>>1, A<<1, B>>1 and B<<1 shifts (Test 28-31); Shift_Flag = 1.

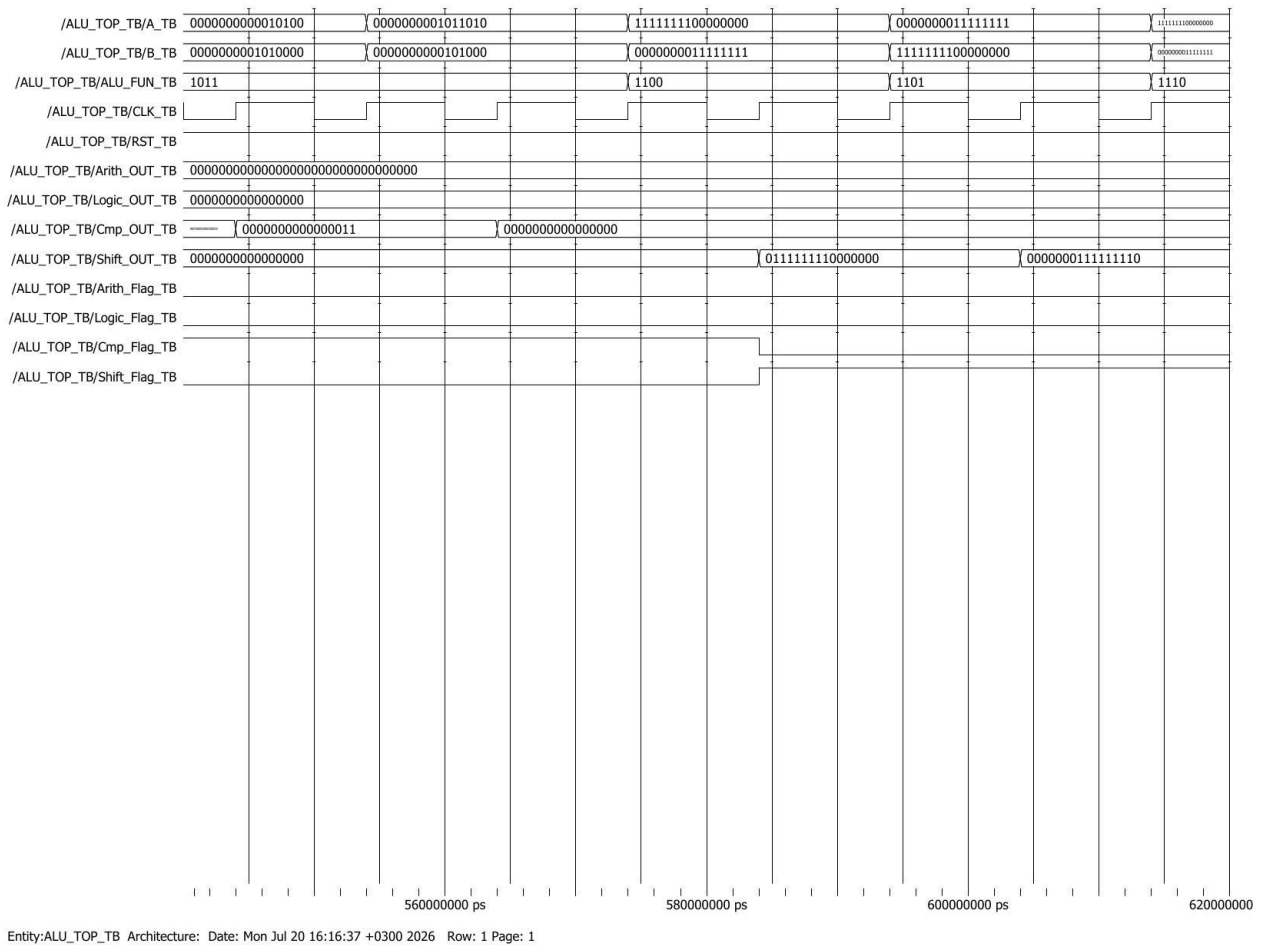


Figure 9: Shift Operations (540 - 620 us), ALU_FUN = 11xx

4.3 Full Simulation Log

Console transcript produced by ALU_TOP_TB.v, showing every PASS result and the final tally (31/31 passed, 0 failed):

```
# =====
#     ALU_TOP Testbench Started
#     Clock: 100KHz (40% low, 60% high)
# =====
#
# --- Signed Arithmetic: Addition (ALU_FUN=0000) ---
# TEST 1 PASSED: ADD: NEG + NEG | Arith_OUT = 4294967282 (0xffffffff2), Flags = 0001
# TEST 2 PASSED: ADD: POS + NEG | Arith_OUT = 10 (0x0000000a), Flags = 0001
# TEST 3 PASSED: ADD: NEG + POS | Arith_OUT = 11 (0x0000000b), Flags = 0001
# TEST 4 PASSED: ADD: POS + POS | Arith_OUT = 55 (0x00000037), Flags = 0001
#
# --- Signed Arithmetic: Subtraction (ALU_FUN=0001) ---
# TEST 5 PASSED: SUB: NEG - NEG | Arith_OUT = 6 (0x00000006), Flags = 0001
# TEST 6 PASSED: SUB: POS - NEG | Arith_OUT = 30 (0x0000001e), Flags = 0001
# TEST 7 PASSED: SUB: NEG - POS | Arith_OUT = 4294967277 (0xffffffff), Flags = 0001
# TEST 8 PASSED: SUB: POS - POS | Arith_OUT = 30 (0x0000001e), Flags = 0001
#
# --- Signed Arithmetic: Multiplication (ALU_FUN=0010) ---
# TEST 9 PASSED: MUL: NEG * NEG | Arith_OUT = 40 (0x00000028), Flags = 0001
# TEST 10 PASSED: MUL: POS * NEG | Arith_OUT = 4294967286 (0xffffffff6), Flags = 0001
# TEST 11 PASSED: MUL: NEG * POS | Arith_OUT = 4294967282 (0xffffffff2), Flags = 0001
# TEST 12 PASSED: MUL: POS * POS | Arith_OUT = 48 (0x00000030), Flags = 0001
#
# --- Signed Arithmetic: Division (ALU_FUN=0011) ---
```

```

# TEST 13 PASSED: DIV: NEG / NEG | Arith_OUT = 2 (0x00000002), Flags = 0001
# TEST 14 PASSED: DIV: POS / NEG | Arith_OUT = 4294967281 (0xfffffffff1), Flags = 0001
# TEST 15 PASSED: DIV: NEG / POS | Arith_OUT = 4294967292 (0xfffffffffc), Flags = 0001
# TEST 16 PASSED: DIV: POS / POS | Arith_OUT = 25 (0x00000019), Flags = 0001
#
# --- Logic Operations (ALU_FUN=01xx) ---
# TEST 17 PASSED: LOGIC: AND | Logic_OUT = 0x0f00, Flags = 0010
# TEST 18 PASSED: LOGIC: OR | Logic_OUT = 0xff0f, Flags = 0010
# TEST 19 PASSED: LOGIC: NAND | Logic_OUT = 0xf0ff, Flags = 0010
# TEST 20 PASSED: LOGIC: NOR | Logic_OUT = 0x00f0, Flags = 0010
#
# --- Compare Operations (ALU_FUN=10xx) ---
# TEST 21 PASSED: CMP: NOP | Cmp_OUT = 0, Flags = 0100
# TEST 22 PASSED: CMP: A == B (True) | Cmp_OUT = 1, Flags = 0100
# TEST 23 PASSED: CMP: A == B (False) | Cmp_OUT = 0, Flags = 0100
# TEST 24 PASSED: CMP: A > B (True) | Cmp_OUT = 2, Flags = 0100
# TEST 25 PASSED: CMP: A > B (False) | Cmp_OUT = 0, Flags = 0100
# TEST 26 PASSED: CMP: A < B (True) | Cmp_OUT = 3, Flags = 0100
# TEST 27 PASSED: CMP: A < B (False) | Cmp_OUT = 0, Flags = 0100
#
# --- Shift Operations (ALU_FUN=11xx) ---
# TEST 28 PASSED: SHIFT: A >> 1 | Shift_OUT = 0x7f80, Flags = 1000
# TEST 29 PASSED: SHIFT: A << 1 | Shift_OUT = 0x01fe, Flags = 1000
# TEST 30 PASSED: SHIFT: B >> 1 | Shift_OUT = 0x007f, Flags = 1000
# TEST 31 PASSED: SHIFT: B << 1 | Shift_OUT = 0xfe00, Flags = 1000
#
# =====
#      Testbench Complete
#      Total Tests: 31
#      Passed: 31
#      Failed: 0
# =====
# ALL TESTS PASSED!
# ** Note: $finish      : Assignments/Assignment 4/4.0/ALU_TOP_TB.v(357)
#      Time: 646500 ns  Iteration: 0  Instance: /ALU_TOP_TB

```

Log 1: Full ModelSim transcript for ALU_TOP_TB

05 DISCUSSION

All 31 directed test cases passed on the first fully-debugged run, matching the expected values computed by hand/spreadsheet for each sign combination. The reset/initialization window confirms the asynchronous active-low reset correctly clears A, B, ALU_FUN and every registered output and flag before stimulus begins. Because each functional unit only updates its output while its own block-enable is high, the flag outputs behave exactly as specified: Arith_Flag, Logic_Flag, CMP_Flag and SHIFT_Flag are mutually exclusive and only the block corresponding to ALU_FUNC[3:2] ever drives a non-zero result. Signed multiplication and division were the operations most sensitive to operand declaration - defining A and B as signed inside Arithmetic_Unit, together with signed testbench operands (two's-complement literals), was necessary for the results to match expected signed arithmetic rather than unsigned wrap-around values. No timing violations or mismatches were observed across the 620 us simulation window.

06 CONCLUSION

A 16-bit ALU_TOP was designed as five cooperating Verilog modules (Decoder_Unit, Arithmetic_Unit, Logic_Unit, Cmp_Unit, Shift_Unit) and verified with a 31-case self-checking testbench at a 100 kHz, 40/60 duty-cycle clock. All test cases passed, confirming correct signed arithmetic, Boolean logic, comparison and shift behaviour, correct block-enable decoding from ALU_FUNC[3:2], correct operation selection from ALU_FUNC[1:0], and correct asynchronous active-low reset behaviour on every registered output and flag.